

Филитов Миша

АКОС 4

Язык Ассемблера

НИУ ВШЭ ФКН ПМИ 2026

Зачем?

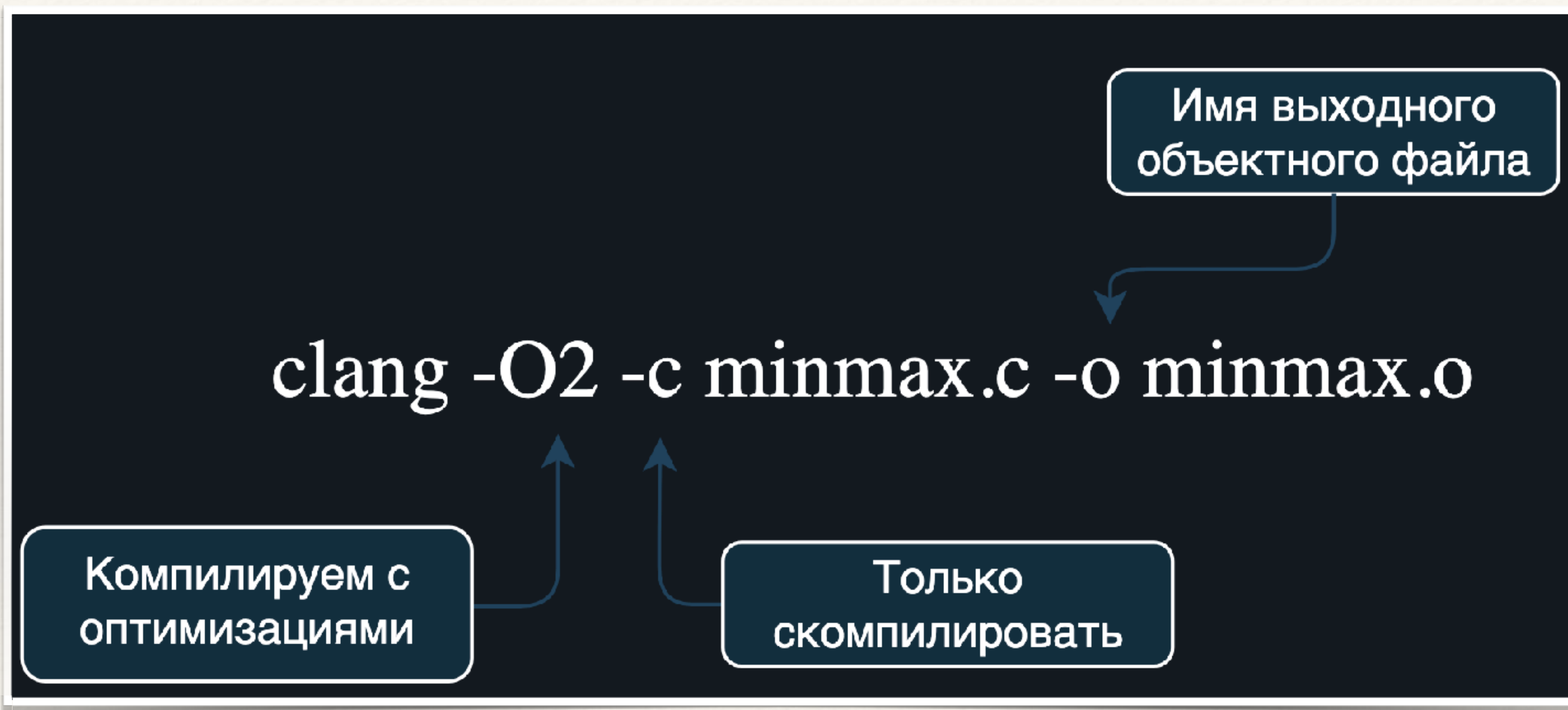
- ❖ ~~Писать код на языке ассемблера~~
- ❖ Понимать, во что компилируется и как исполняется код
- ❖ Понимать, что делают указатели, функции, стек и память
- ❖ Уметь понять, где упала программа и почему
- ❖ Понимать, как работают оптимизации

Как вообще выглядит то, что исполняет
процессор?

Скомпилируем простую функцию

```
1 int minmax(int a, int b) {  
2     int min = b;  
3     if (a < b) {  
4         min = a;  
5     }  
6     return min;  
7 }
```

Скомпилируем простую функцию



Смотрим на получившийся бинарь

```
1  int minmax(int a, int b) {  
  1    int min = b;  
  2    if (a < b) {  
  3        min = a;  
  4    }  
  5    return min;  
  6 }  
  7  
  8  
~  
~  
~  
~
```

```
→ class4 objdump -d -Mintel minmax.o  
  
minmax.o:      file format elf64-x86-64  
  
Disassembly of section .text:  
  
0000000000000000 <minmax>:  
    0:  89 f0          mov     eax,esi  
    2:  39 f7          cmp     edi,esi  
    4:  0f 4c c7      cmovl   eax,edi  
    7:  c3            ret  
  
→ class4 █
```

<https://godbolt.org/z/bWGz6rWY9>

Демо

Декодирование инструкций



Пруф

Opcode	Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
3C ib	CMP AL, imm8	I	Valid	Valid	Compare imm8 with AL.
3D iw	CMP AX, imm16	I	Valid	Valid	Compare imm16 with AX.
3D id	CMP EAX, imm32	I	Valid	Valid	Compare imm32 with EAX.
REX.W + 3D id	CMP RAX, imm32	I	Valid	N.E.	Compare imm32 sign-extended to 64-bits with RAX.
80 /7 ib	CMP r/m8, imm8	MI	Valid	Valid	Compare imm8 with r/m8.
REX + 80 /7 ib	CMP r/m8*, imm8	MI	Valid	N.E.	Compare imm8 with r/m8.
81 /7 iw	CMP r/m16, imm16	MI	Valid	Valid	Compare imm16 with r/m16.
81 /7 id	CMP r/m32, imm32	MI	Valid	Valid	Compare imm32 with r/m32.
REX.W + 81 /7 id	CMP r/m64, imm32	MI	Valid	N.E.	Compare imm32 sign-extended to 64-bits with r/m64.
83 /7 ib	CMP r/m16, imm8	MI	Valid	Valid	Compare imm8 with r/m16.
83 /7 id	CMP r/m32, imm8	MI	Valid	Valid	Compare imm8 with r/m32.
REX.W + 83 /7 id	CMP r/m64, imm8	MI	Valid	N.E.	Compare imm8 with r/m64.
38 /r	CMP r/m8, r8	MR	Valid	Valid	Compare r8 with r/m8.
REX + 38 /r	CMP r/m8*, r8*	MR	Valid	N.E.	Compare r8 with r/m8.
39 /r	CMP r/m16, r16	MR	Valid	Valid	Compare r16 with r/m16.
39 /r	CMP r/m32, r32	MR	Valid	Valid	Compare r32 with r/m32.
REX.W + 39 /r	CMP r/m64, r64	MR	Valid	N.E.	Compare r64 with r/m64.
3A /r	CMP r8, r/m8	RM	Valid	Valid	Compare r/m8 with r8.
REX + 3A /r	CMP r8*, r/m8*	RM	Valid	N.E.	Compare r/m8 with r8.
3B /r	CMP r16, r/m16	RM	Valid	Valid	Compare r/m16 with r16.
3B /r	CMP r32, r/m32	RM	Valid	Valid	Compare r/m32 with r32.
REX.W + 3B /r	CMP r64, r/m64	RM	Valid	N.E.	Compare r/m64 with r64.

<https://www.felixcloutier.com/x86/cmp>

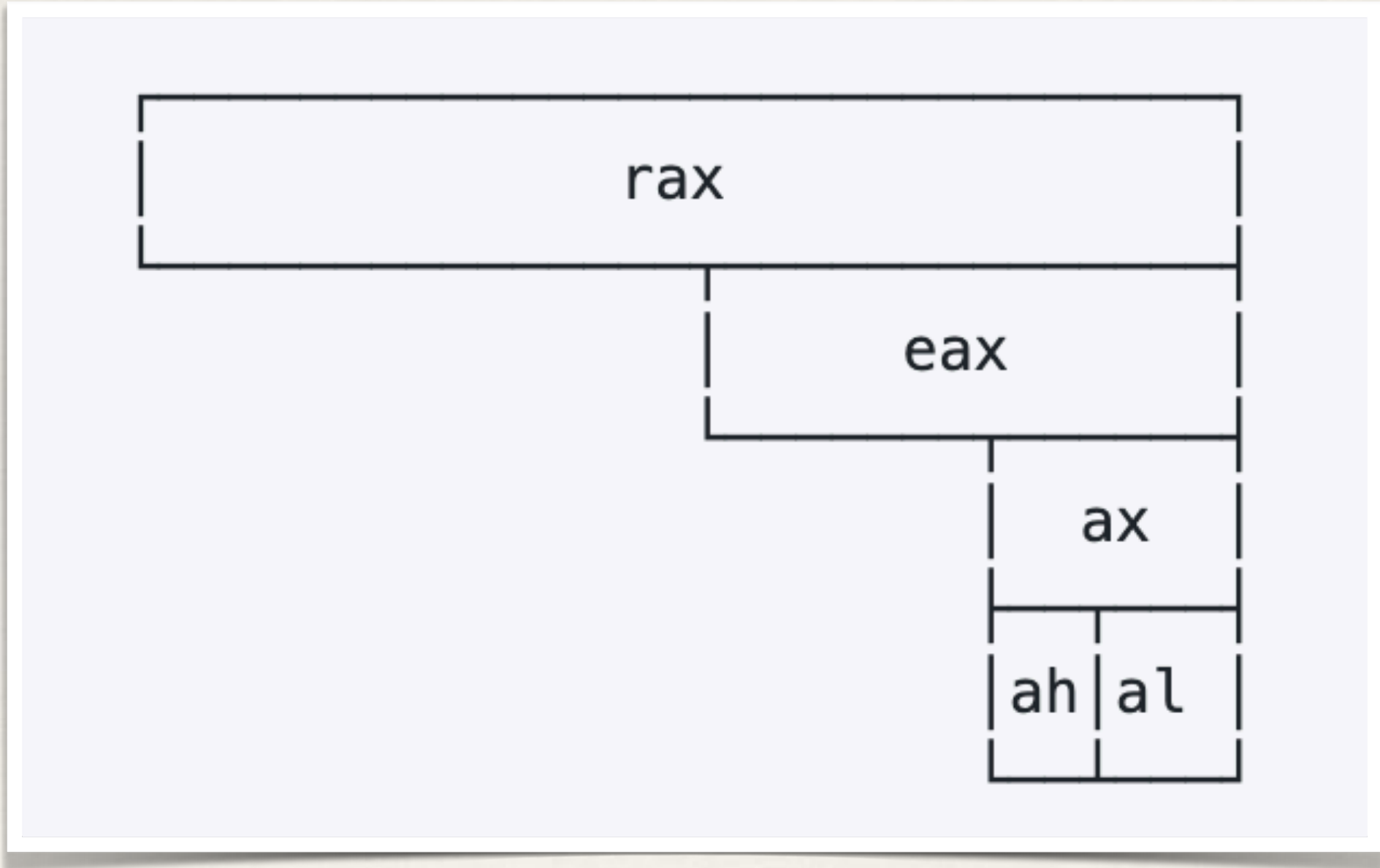
Table 2-2. 32-Bit Addressing Forms with the ModR/M Byte

r8(/r) r16(/r) r32(/r) mm(/r) xmm(/r) (In decimal) /digit (Opcode) (In binary) REG =			AL AX EAX MM0 XMM0 0 000	CL CX ECX MM1 XMM1 1 001	DL DX EDX MM2 XMM2 2 010	BL BX EBX MM3 XMM3 3 011	AH SP ESP MM4 XMM4 4 100	CH BP EBP MM5 XMM5 5 101	DH SI ESI MM6 XMM6 6 110	BH DI EDI MM7 XMM7 7 111
Effective Address	Mod	R/M	Value of ModR/M Byte (in Hexadecimal)							
[EAX] [ECX] [EDX] [EBX] [---]1 disp322	00	000 001 010 011 100 101 110 111	00 01 02 03 04 05 06 07	08 09 0A 0B 0C 0D 0E 0F	10 11 12 13 14 15 16 17	18 19 1A 1B 1C 1D 1E 1F	20 21 22 23 24 25 26 27	28 29 2A 2B 2C 2D 2E 2F	30 31 32 33 34 35 36 37	38 39 3A 3B 3C 3D 3E 3F
[EAX]+disp83 [ECX]+disp8 [EDX]+disp8 [EBX]+disp8 [---]+disp8 [EBP]+disp8 [ESI]+disp8 [EDI]+disp8	01	000 001 010 011 100 101 110 111	40 41 42 43 44 45 46 47	48 49 4A 4B 4C 4D 4E 4F	50 51 52 53 54 55 56 57	58 59 5A 5B 5C 5D 5E 5F	60 61 62 63 64 65 66 67	68 69 6A 6B 6C 6D 6E 6F	70 71 72 73 74 75 76 77	78 79 7A 7B 7C 7D 7E 7F
[EAX]+disp32 [ECX]+disp32 [EDX]+disp32 [EBX]+disp32 [---]+disp32 [EBP]+disp32 [ESI]+disp32 [EDI]+disp32	10	000 001 010 011 100 101 110 111	80 81 82 83 84 85 86 87	88 89 8A 8B 8C 8D 8E 8F	90 91 92 93 94 95 96 97	98 99 9A 9B 9C 9D 9E 9F	A0 A1 A2 A3 A4 A5 A6 A7	A8 A9 AA AB AC AD AE AF	B0 B1 B2 B3 B4 B5 B6 B7	B8 B9 BA BB BC BD BE BF
EAX/AX/AL/MM0/XMM0 ECX/CX/CL/MM/XMM1 EDX/DX/DL/MM2/XMM2 EBX/BX/BL/MM3/XMM3 ESP/SP/AH/MM4/XMM4 EBP/BP/CH/MM5/XMM5 ESI/SI/DH/MM6/XMM6 EDI/DI/BH/MM7/XMM7	11	000 001 010 011 100 101 110 111	C0 C1 C2 C3 C4 C5 C6 C7	C8 C9 CA CB CC CD CE CF	D0 D1 D2 D3 D4 D5 D6 D7	D8 D9 DA DB DC DD DE DF	E0 E1 E2 E3 E4 E5 E6 E7	E8 E9 EA EB EC ED EE EF	F0 F1 F2 F3 F4 F5 F6 F7	F8 F9 FA FB FC FD FE FF

<https://cdrdv2-public.intel.com/812383/253666-sdm-vol-2a.pdf>

Регистр

Ячейка памяти внутри процессора. На x86_64 размера 64 бита.



64-бит	32-бит	16-бит	8-бит	8-бит (старшие)
rax	eax	ax	al	ah
rcx	ecx	cx	cl	ch
rdx	edx	dx	dl	dh
rbx	ebx	bx	bl	bh
rsi	esi	si	sil	
rdi	edi	di	dil	
rsp	esp	sp	spl	
rbp	ebp	bp	bpl	
r8	r8d	r8w	r8b	
r9	r9d	r9w	r9b	
r10	r10d	r10w	r10b	
r11	r11d	r11w	r11b	
r12	r12d	r12w	r12b	
r13	r13d	r13w	r13b	
r14	r14d	r14w	r14b	
r15	r15d	r15w	r15b	

Инструкции

- ❖ **DST** - destination - куда сохранить значение
- ❖ **SRC** - source - откуда взять значение

add	DST, SRC	/* DST += SRC */
sub	DST, SRC	/* DST -= SRC */
inc	DST	/* ++DST */
dec	DST	/* --DST */
neg	DST	/* DST = -DST */
mov	DST, SRC	/* DST = SRC */
imul	SRC	/* (eax,edx) = eax * SRC – знаковое */
mul	SRC	/* (eax,edx) = eax * SRC – беззнаковое */
and	DST, SRC	/* DST &= SRC */
or	DST, SRC	/* DST = SRC */
xor	DST, SRC	/* DST ^= SRC */
not	DST	/* DST = ~DST */
cmp	DST, SRC	/* DST – SRC, результат не сохраняется, */
test	DST, SRC	/* DST & SRC, результат не сохраняется */
adc	DST, SRC	/* DST += SRC + CF */
sbb	DST, SRC	/* DST -= SRC – CF */

simpleio

- ❖ readi32 - считать 32б число (сложится в eax)
- ❖ readi64 - считать 64б число (сложится в rax)
- ❖ writei32 - напечатать 32б число (печатается из edi)
- ❖ writei64 - напечатать 64б число (печатается из rdi)
- ❖ Все библиотечные функции не меняют значения регистров rsp, rbp, rbx, r12, r13, r14, r15.
- ❖ Если ввод закончен, что флаг CF выставится в 1. Проверить по jc.
- ❖ Чтобы закончить ввод нужно нажать CTRL+D или произвести ввод из файла cat file.txt | ./a.out или через echo "1 2 3" | ./a.out

```
8 // Все библиотечные функции не меняют
7 // значения регистров
6 // rsp, rbp, rbx, r12, r13, r14, r15
5     .intel_syntax noprefix
4     .global main
3
2 main:
1     // Результат в rax
10    call    readi64
1     // mov dst, src  dst = src
2     mov     rdi, rax
3
4     // Пишем из rdi
5     call    writei64
6
7     call    finish
read_write.S                                10,19                                Bot

> class4 gcc read_write.S simpleio_x86_64.S
> class4 ./a.out
55
55
```

eflags

Большинство арифметических инструкций в результате вычисления результата инструкции устанавливают арифметические флаги слова состояния процесса.

- ❖ Флаг ZF устанавливается, если в результате операции был получен нуль.
- ❖ Флаг SF устанавливается, если в результате операции было получено отрицательное число.
- ❖ Флаг CF устанавливается, если в результате выполнения операции произошёл перенос из старшего бита результата. Например, для сложения CF устанавливается если результат сложения двух беззнаковых чисел не может быть представлен 32-битным беззнаковым числом.
- ❖ Флаг OF устанавливается, если в результате выполнения операции произошло переполнение знакового результата. Например, при сложении OF устанавливается, если результат сложения двух знаковых чисел не может быть представлен 32-битным знаковым числом.

	Unsigned	Signed
000	0	0
001	1	1
010	2	2
011	3	3
100	4	-4
101	5	-3
110	6	-2
111	7	-1

eflags

- ❖ **ZF:** 100 + 100
- ❖ **CF:** 110 + 111
- ❖ **SF:** 001 + 100
- ❖ **OF:** 011 + 010
- ❖ **CF, OF:** 100 + 111

	Unsigned	Signed
000	0	0
001	1	1
010	2	2
011	3	3
100	4	-4
101	5	-3
110	6	-2
111	7	-1

Управление выполнением

```
jz    label    /* переход, если равно (нуль), ZF == 1 */
jnz   label    /* переход, если не равно (не нуль), ZF == 0 */
jc    label    /* переход, если CF == 1 */
jnc   label    /* переход, если CF == 0 */
jo    label    /* переход, если OF == 1 */
jno   label    /* переход, если OF == 0 */
jg    label    /* переход, если больше для знаковых чисел ZF == 0 && SF == 0F */
jge   label    /* переход, если >= для знаковых чисел SF == 0F */
jl    label    /* переход, если < для знаковых чисел SF != 0F */
jle   label    /* переход, если <= для знаковых чисел ZF == 1 || SF != 0F */
ja    label    /* переход, если > для беззнаковых чисел ZF == 0 && CF == 0 */
jae   label    /* переход, если >= (беззнаковый) CF == 0 */
jb    label    /* переход, если < (беззнаковый) CF == 1 */
jbe   label    /* переход, если <= (беззнаковый) ZF == 1 || CF == 1 */
```

```
31 main:
32     // Результат в rax
33     call    readi64
34     // первое число
35     mov     r12, rax
36
37     // Результат в rax
38     call    readi64
39     // второе число
40     mov     r13, rax
41
42 my_sub:
43     // consider cmp
44     sub     r12, r13
45     jz      end
46     jl      print_less
47     jmp     print_greater
48
49 print_greater:
50     mov     rdi, 1
51     call    writei64
52     jmp     main
53
54 print_less:
55     mov     rdi, -1
56     call    writei64
57     jmp     main
58
59 end:
60     call    finish
```

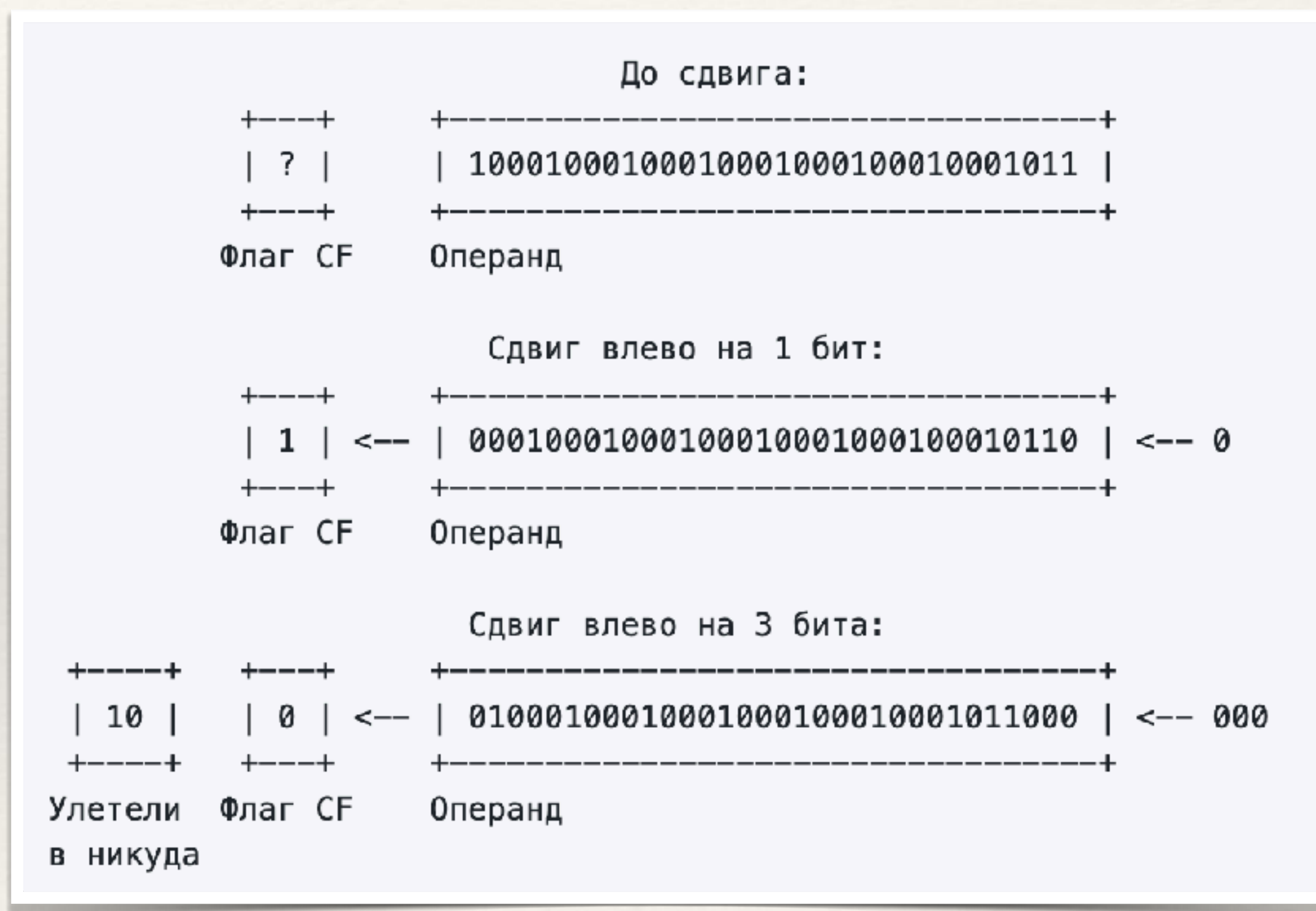
GDB - GNU Debugger

- ❖ **[l]ist** - показать код. 1 3 показать код с третьей строки
- ❖ **[b]reak** - установить брейкпоинт. Нужно указать имя функции или строку. b main. Выполнение дойдет до этого места и остановится.
- ❖ **[r]un** - запустить программу
- ❖ **info break (i b)** - посмотреть брейкпоинты
- ❖ **del <breakpoint num>** - удалить брейкпоинт по номеру
- ❖ **layout next (la next)** - включить следующий layout
- ❖ **layout regs (la reg)** - включить просмотр регистров
- ❖ **[s]tep** - сделать шаг исполнения (с заходом в функции)
- ❖ **[n]ext** - сделать шаг исполнения (без захода в функции)
- ❖ **[c]ontinue** - продолжить выполнение до следующего брейкпоинта
- ❖ **refresh (ctrl+l)** - обновить отображение (если все съехало или перекособочилось)
- ❖ **[f]inish** - выйти из функции
- ❖ **help <command>** - справка по команде
- ❖ **p/t \$rax** - напечатать регистр eax бинарно
- ❖ **p/z \$rax** - напечатать регистр eax 16-рично
- ❖ **set \$rax=5** - установить значение \$eax равное 5
- ❖ **jump main** - продолжить выполнение с другого места (не обязательно main, можно указать любое другое место или номер строки)
- ❖ **p/t \$rax^3** - напечатать в бинарном виде результат выполнения команды \$eax XOR 3 (другие логические операции тоже можно использовать)
- ❖ **ctrl+x o** - переключиться между панелями (чтобы стрелка вверх работала)

Сдвиги

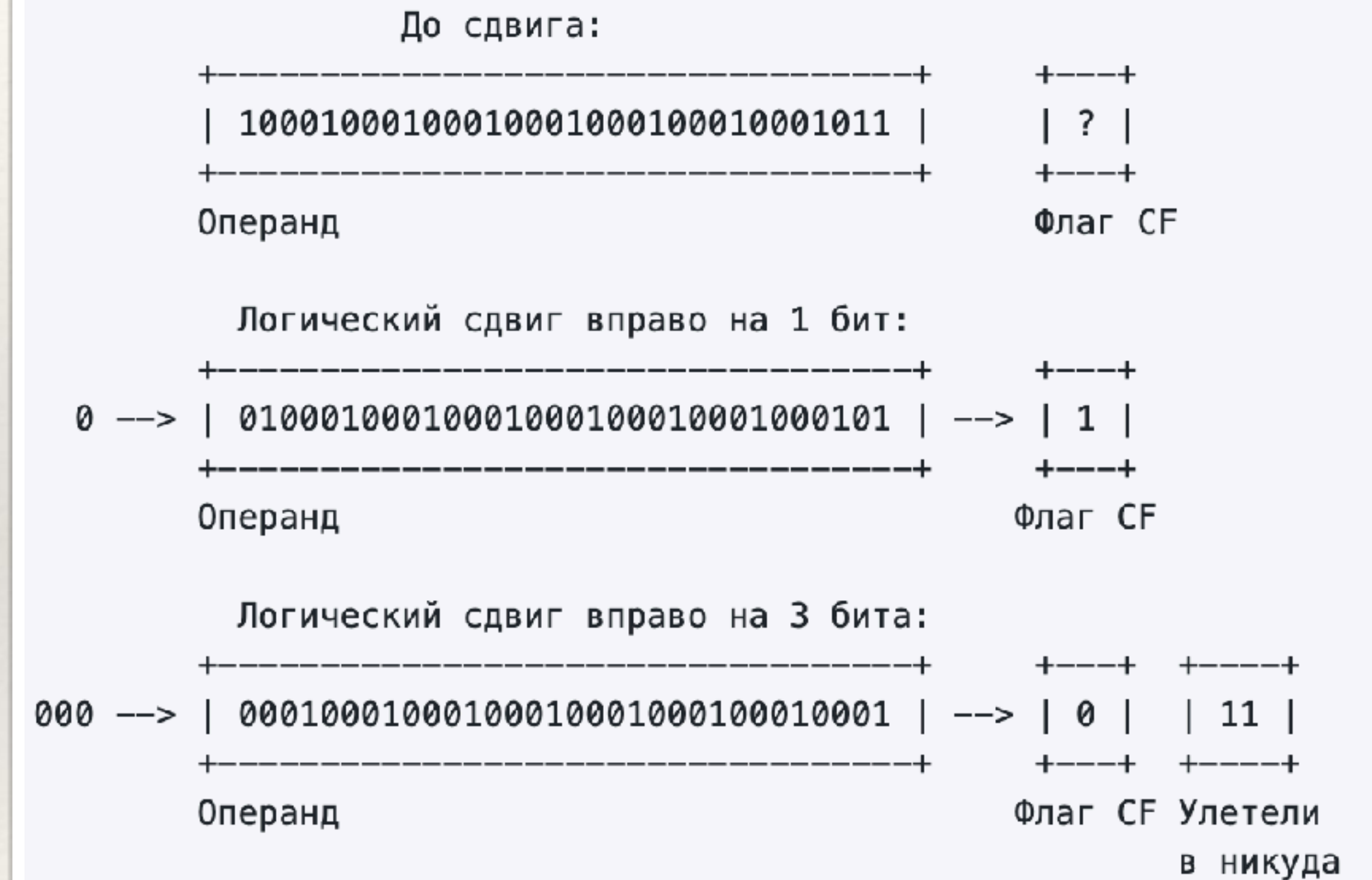
shl – логический сдвиг влево

- ❖ Сдвигает все биты регистра влево
- ❖ Последний сдвинутый бит находится в CF
- ❖ Умножение на 2
- ❖ `shl r12, 3` - умножить r12 на 8



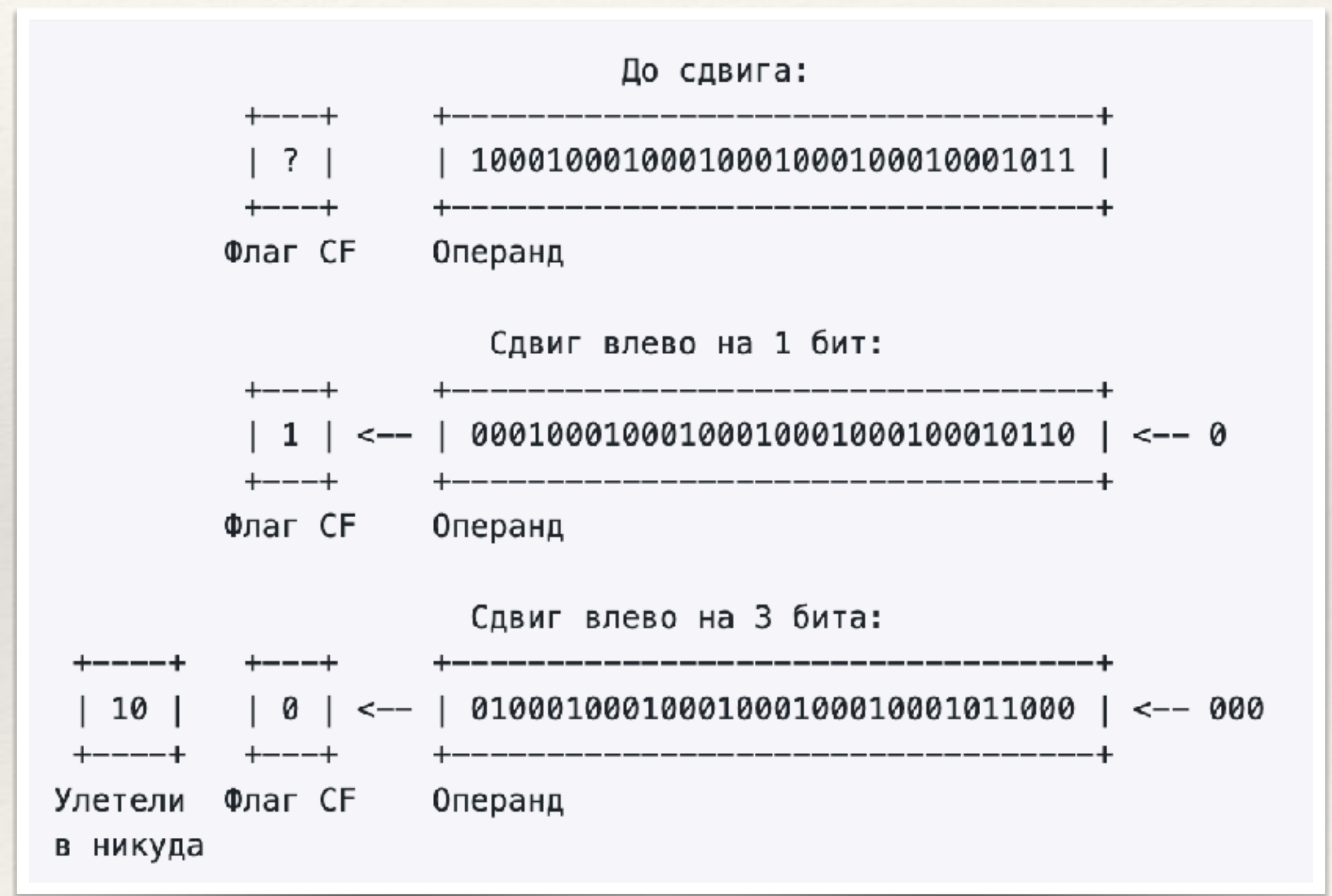
shr - логический сдвиг вправо

- ❖ Сдвигает все биты регистра вправо
- ❖ Последний сдвинутый бит находится в CF
- ❖ Деление на 2
- ❖ `shr r12, 3` - поделить r12 на 8



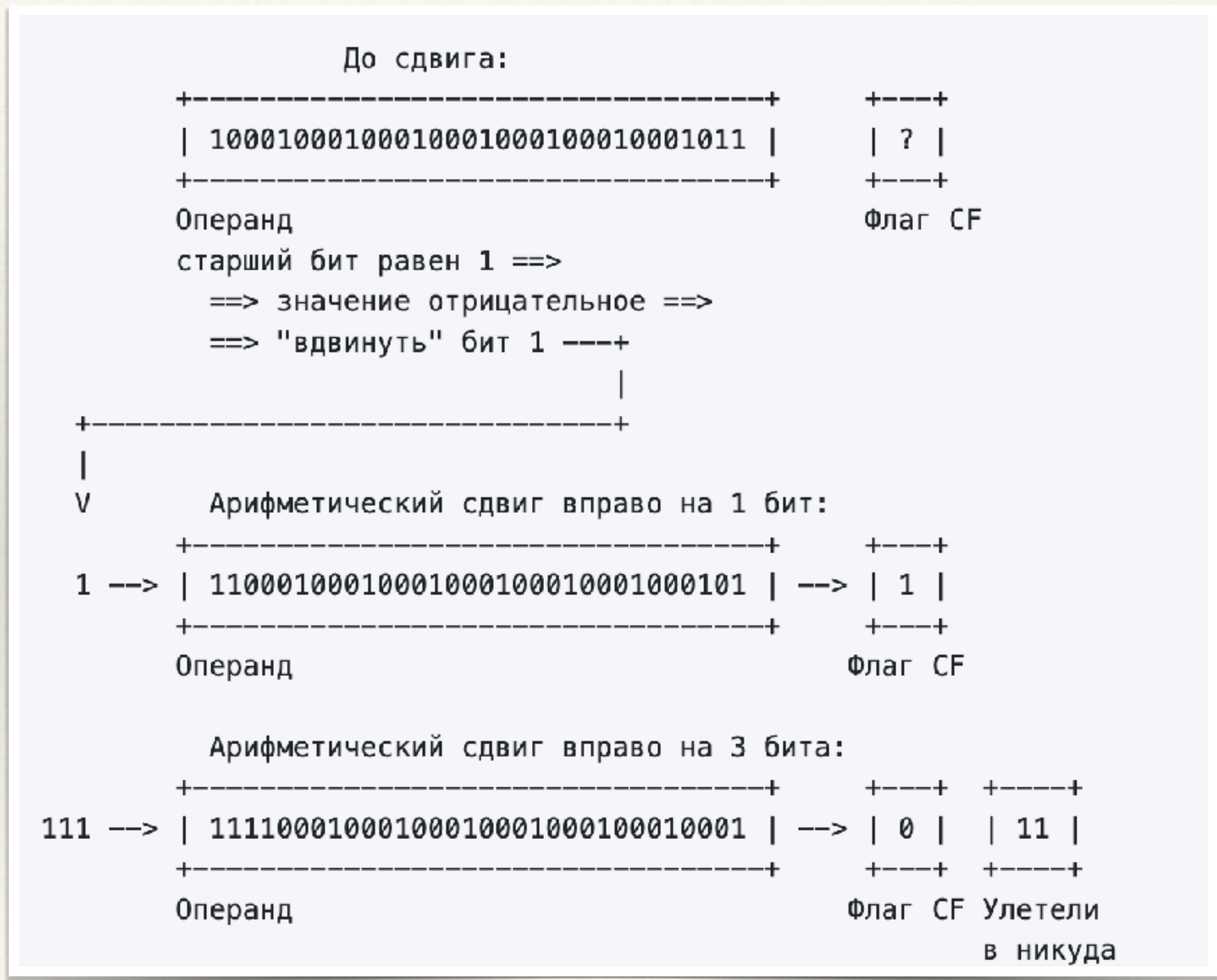
sal - арифметический сдвиг влево

- ❖ Сдвигает все биты регистра влево
- ❖ Последний сдвинутый бит находится в CF
- ❖ Умножение на 2
- ❖ `sal r12, 3` - умножить r12 на 8
- ❖ Не отличается от `shl`, но мы будем требовать, чтобы использовался в корректном контексте



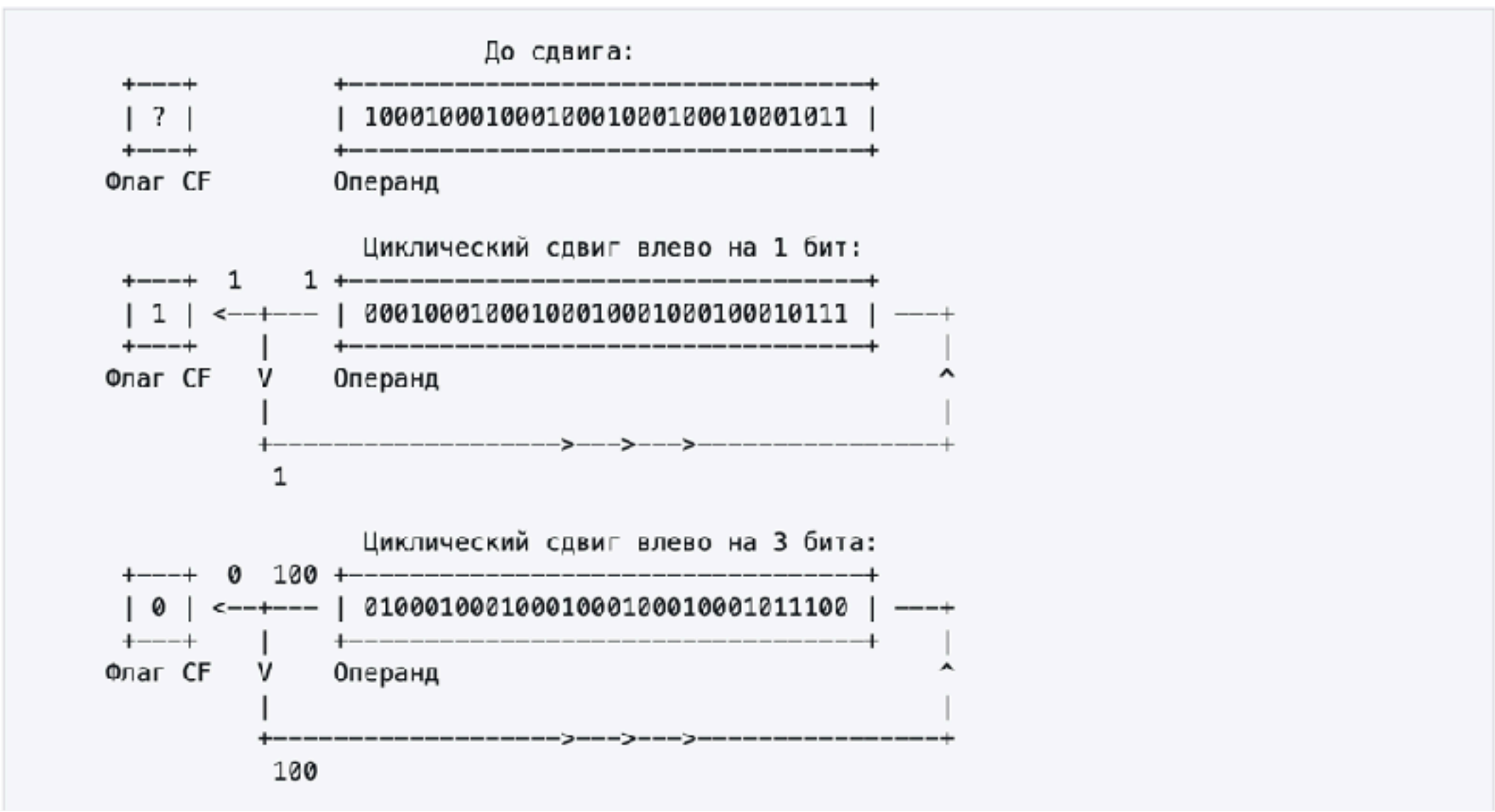
sar – арифметический сдвиг вправо

- ❖ Знаково сдвигает все биты регистра вправо
- ❖ Последний сдвинутый бит находится в CF
- ❖ Деление на 2
- ❖ **sar r12, 3** - поделить r12 на 8 с учетом знака



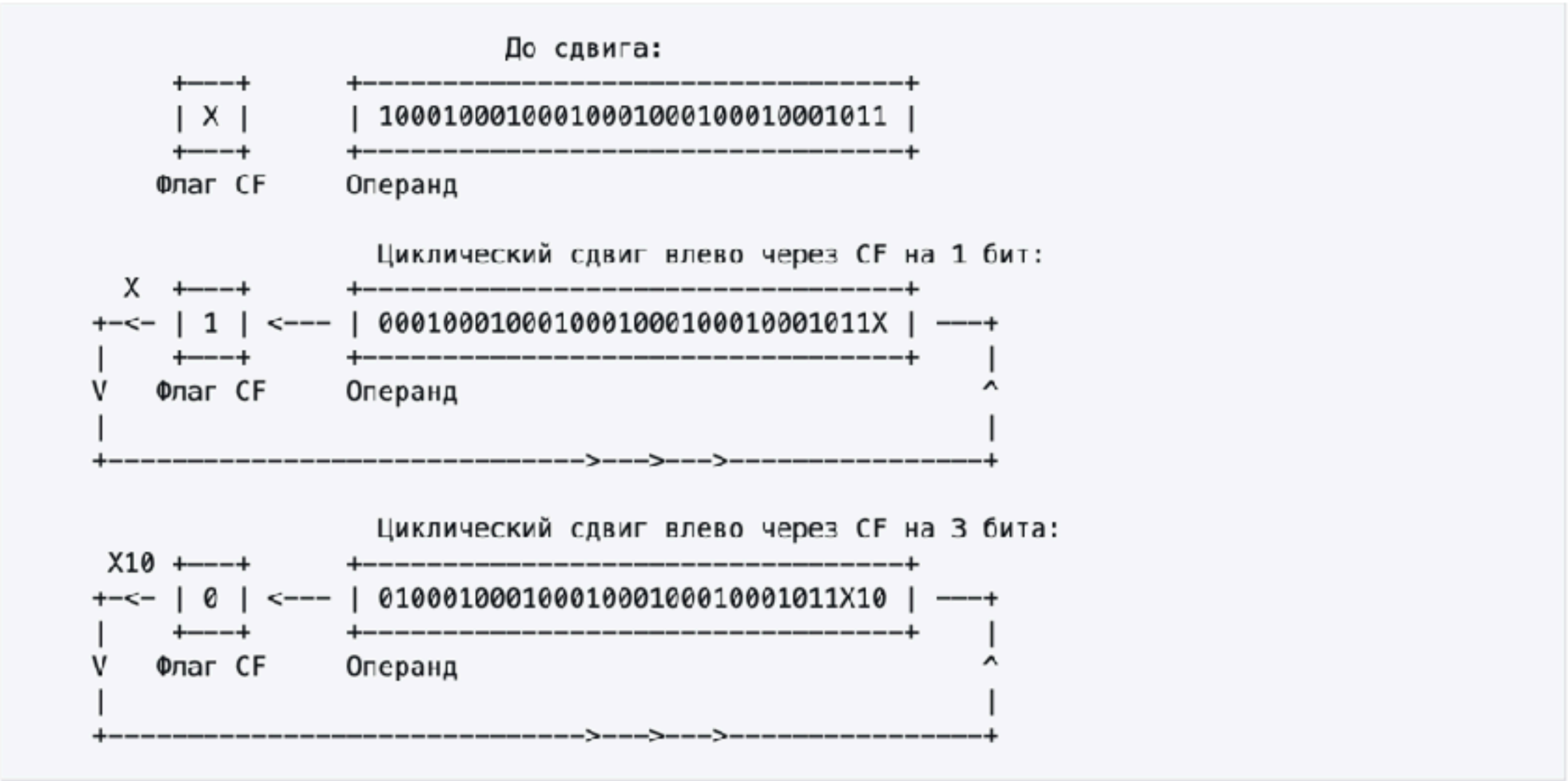
Другие прикольные сдвиги

Принцип работы команды `rol`:



[Вики](#)

Принцип работы команды `rcl`:



[Вики](#)

Следующее занятие

- ❖ Секции кода
- ❖ Память и стек
- ❖ Соглашение о вызовах
- ❖ Системные вызовы
- ❖ Подпрограммы